GITHUB: https://github.com/eddie-edu/CMPSC445_Project1

Project Description (5%)

This project makes the machine learning pipeline that predicts the value-retention ratio of used vehicles and identifies the factors that influence vehicle depreciation the most. It predicts how much of a car's original price value is kept based on features(mileage, age, make, if there were any accidents, etc). I targeted 4 companies, each with 3-4 models:

1. Toyota: Camry, Corolla, RAV4, Tacoma
2. Honda: Civic, Accord, CR-V
3. Ford: F-150, Escape, Explorer
4. Chevy: Silverado 1500, Equinox, Malibu

These models were picked due to the abundance of them on the market, making it less effort to scrape.

Data Collection with discussion(5%)

I picked 3 sources: Craigslist and TrueCar for the used car listings, and Kelley Blue Book for the new-car(MSRP) values. 1,185 entries were TrueCar and 975 were from Craigslist, well over the 500 you asked for. Kelley Blue Book's bot protection blocked Python's requests library because of its TLS fingerprint, so I switched to curl_cffi, which mimics Chrome, Firefox, or Safari. These were picked due to them being more scraper-friendly while still having an abundance of listings. Even then, TrueCar ended up blocking my IP entirely almost an hour later, and still has not been lifted after 3 days. Luckily, I have it where every page the scraper downloads is cached, so when TrueCar did block me the raw HTML was saved. This allows me to make any changes to the parser without having to re-scape. For TrueCar and Kelley Blue Book, the program reads JSON instead of the HTML because the data isn't embedded in the HTML like Craigslist, it's in a separate embedded "NEXT_DATA" portion. For Craigslist in particular, their filter is not good, and filtered out cars we were looking for. Instead, the program searched Craigslist by keyword for each model ("camry", "f150") in 16 cities, then filtered out titles naming the wrong model or year.

Preprocessing and Integration with discussion(5%)

Craigslist had the most inaccurate data by far: 116 entries removed as opposed to 21 by TrueCar: all 21 of TrueCar's were from a missing price. There were also some other criteria for entries to remain: as an example people were typing in under 100 miles on a car years old. As an example, someone types 40 on a 3 year old car, that's unrealistic. People naturally assume it's 40,000 miles, but to keep the model accurate I just dropped those rows entirely. Same for prices: 50 dollars for a car makes no sense.

We also normalized the two sources: fields were unified (odometer on Craigslist gets renamed to mileage) and converted to numbers (Craigslist sends numbers like $12,345). Any other invalid fields, such as VINs, are removed without the row being dropped. If, in the future, we were to decode these VINs(they have car features encoded in them), we would also drop those rows entirely.

2023 listings remained after this process. Below is the attribute breakdown of those entries:

	Median	Min	Max
Price	$20,795	$1,850	$94,957
Mileage	63,804	1,873	369,000
Model year	2021	2012	2024
Base MSRP	$27,515	$16,555	$43,895
Retention ratio	0.77	0.07	2.60

Additionally, TrueCar's median listing is a $25,800 car from 2023 with 42,000 miles, and Craigslist's median listing is a $12,700 car from 2017 with 114,000 miles on it.

Feature Engineering with discussion(5%)

The program writes the features used to "features.csv" for visualization and analysis. I also binary-encode certain fields for the model.

I obviously only used the fields both sources provided. Craigslist was a huge limiter here - if I did this project again I would have picked a different source.

Features extracted from the listing text - craigslist only:
Important to note that I replaced all dashes with spaces in the text, as some people use them in between words and others do not.

- one_owner: searched for “one owner”, “1 owner”, “single owner”
- no_accidents: searches for “no accidents", “accident free”, “clean history”
- Needs_work: “needs work”, “as is”, “check engine”, “won’t start”, “will not start”

This is the one spot where Craigslist did shine - TrueCar had no description at all.

Encoding methods used: One-hot encoding, binary encoding, numeric scaling(StandardScaler, for kNN). This turned the 28 columns left into 64, or 164 columns with one-hot encoding. However, most of those 164 columns were mostly 0s, which wastes spaces and hurt the kNN, so I would have removed them outside of this project. Binary encoding was compact enough where this wasn’t a problem.

Model Development with discussion(5%)

Model types used:

- Linear regression
- K-nearest neighbor
- Decision tree

Each model is wrapped in its own Pipeline object, which combines the preprocessing and the model in one object. This is useful for the predict() function, where it preprocesses the input for you, just like the training data.

Because preprocessing lives inside the pipeline object, it learns only from the training data, so nothing from the test set will leak into the training data. I split the data to be 80% training and 20% testing, stratified by the model. Each model was tuned with a 5 fold GridSearchCV, and the best settings for kNN were k = 10 with distance weighting, and a depth of 8 for the decision tree.

Evaluation Results with discussion (5%)

```
=== src.train ===
train 1618 rows, test 405 rows
 model  cv_rmse  test_mae  test_rmse  test_r2                                best_params
linear   0.1220    0.0807     0.1092   0.8636                                        {}
   knn   0.1240    0.0833     0.1117   0.8573 {'n_neighbors': 10, 'weights': 'distance'}
  tree   0.1425    0.1009     0.1371   0.7850    {'max_depth': 8, 'min_samples_leaf': 5}
```

The above are the results for the three models. Out of the three, the linear model performed the best, being able to explain 86.4% of the variance. kNN was very close though, with 85.7%. On average, the predictions are 8.1 percentage points of MSRP off (MAE) / 10.92 percentage points of MSRP off (RMSE). This means that, for example, a car that kept 60% of MSRP is predicted between 52% and 68%. RMSE will always be bigger, as it penalizes larger mistakes more than MAE, but is more sensitive to outliers as a result.

I'm surprised how close all the models are. This can be explained by the depreciation being close to linear in age and mileage, so even the simple models fit well.

Depreciation Analysis (5%)

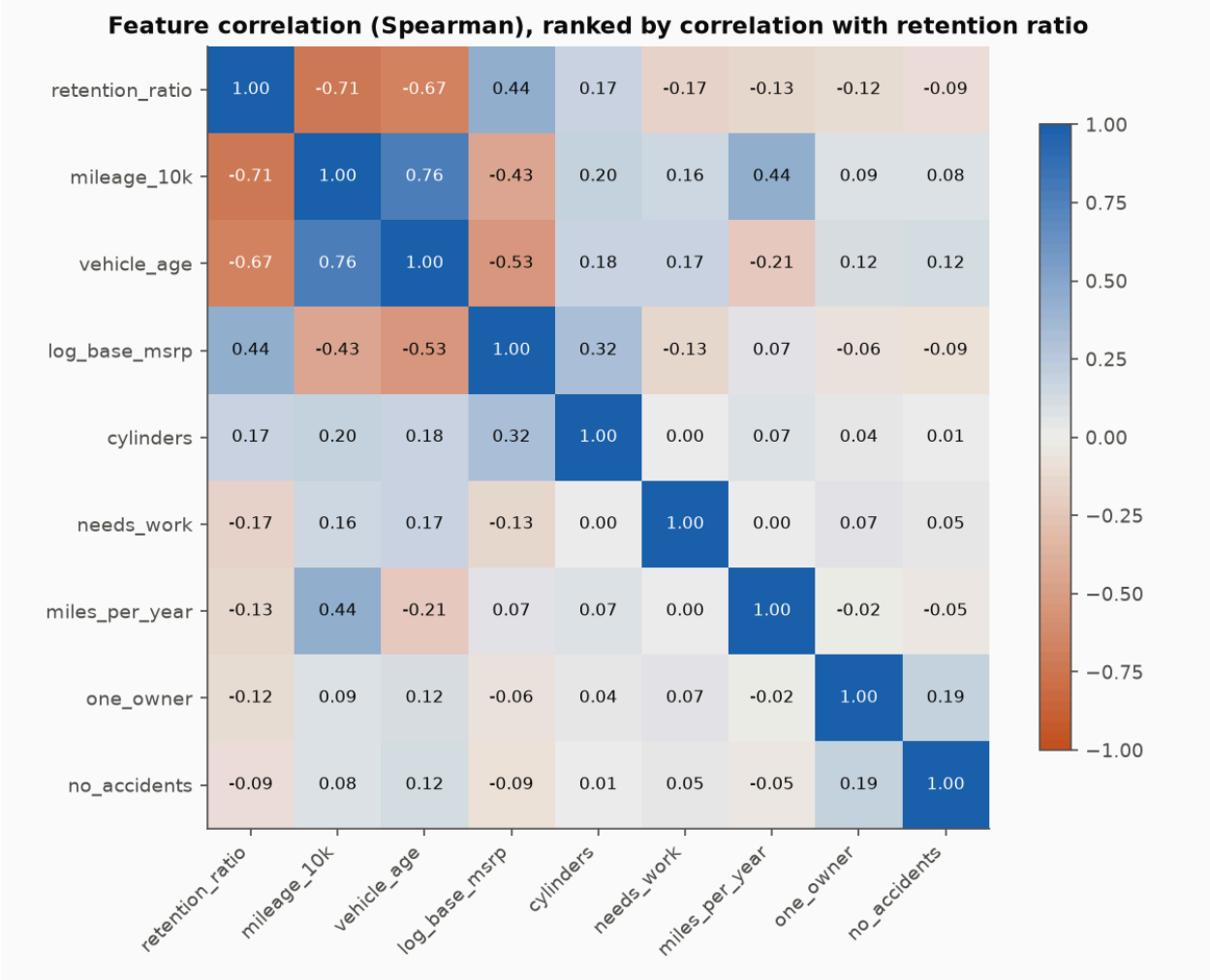
```
mileage_10k       0.575
body_type         0.162
make              0.089
trim              0.053
vehicle_age       0.040
model             0.026
log_base_msrp     0.014
title_status      0.009
state             0.009
seller_type       0.008
drivetrain        0.006
cylinders         0.003
miles_per_year    0.002
fuel_type         0.002
no_accidents      0.000
one_owner         0.000
transmission      0.000
needs_work        0.000
```

The above is based on the decision tree's feature importance. There's a couple of things here I find interesting:

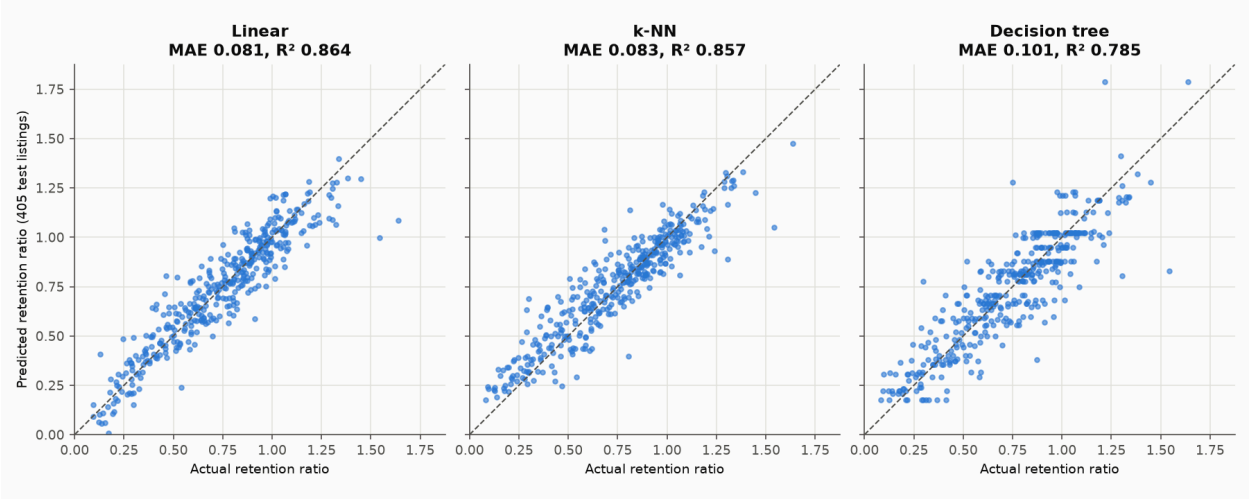
- Mileage accounts for up to 57.5% of the tree's feature importance. This makes sense in the real world, the more a car is driven, the less valuable it is.
- Age is only 4% of the importance. This can be explained by older cars naturally having more mileage, and most cars are consistently driven. Correlated features tend to have just one field take most of the importance.
- All the craigslist description tags rank at 0% importance. This is because there's so few listings that have that information in the description.
- Body type comes in 2nd, which is if the vehicle is a truck or a car.

I picked the decision tree because it is perfect for seeing which features are the most important. Even though it is the least accurate model, it's not behind by much (8% R^2 difference).

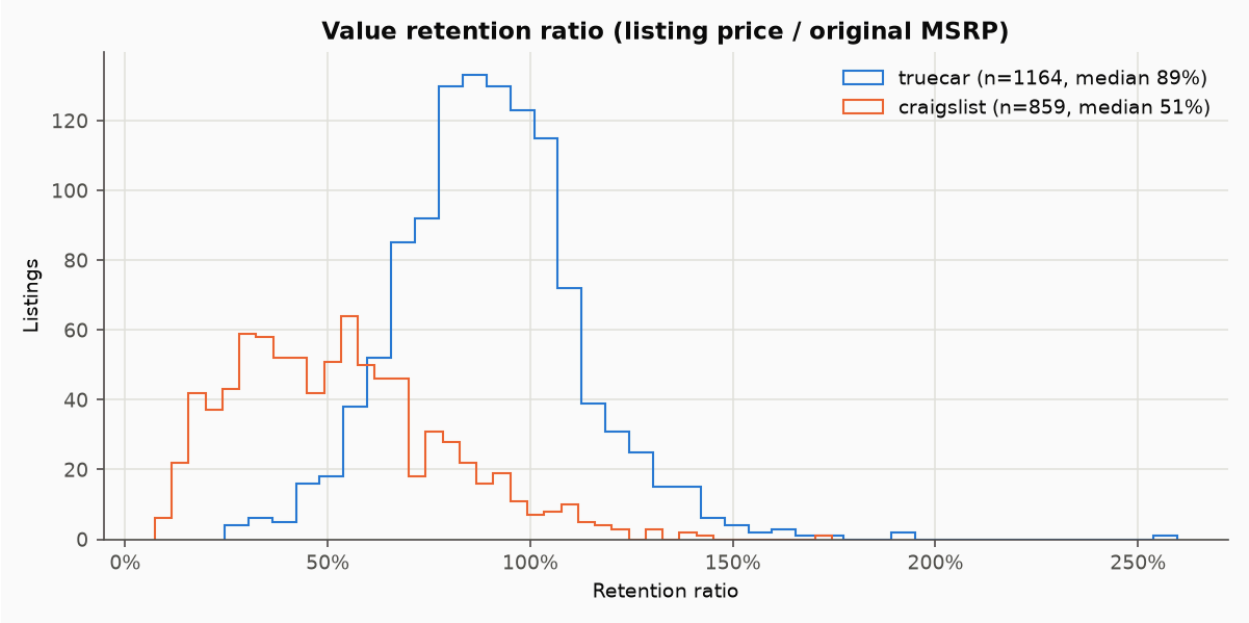
Visualizations with discussion (5%)



This heatmap shows the correlation between features in the data. This further backs up my reasoning on why the vehicle age wasn't as important as expected: the mileage and the vehicle age are the most correlated in the entire dataset. Both vehicle age and mileage have a strong negative correlation to the retention ratio. Miles per year and mileage have a 0.44 correlation, which is actually lower than I expected, but I am not surprised to see them correlated to each other.



This is all the predictions all three models have made. Notice all the horizontal lines that form in the decision tree chart. This is because a regression tree can still output numbers, but only a limited number of numbers. The tree the model uses has a depth of 8, so it can only produce 256 values. So all the cars that fit under one criteria all get the same value. You can also see that Linear and kNN are closer to the diagonal line than decision trees, as they are the more accurate models.



This is meant to measure the difference between the two sources, which are very different. Craigslist is meant for cheaper, more worn-down cars that people just want to get rid of, while TrueCar is more of a formal reselling market, so they get the higher value cars. Interestingly, some listings go over 100% MSRP. This is probably because the original owners got the most expensive variants of those cars, and we took the lowest MSRP when training the model

Conclusion (6%)

Summary of project finding

The depreciation of a car's value is mostly from the age and mileage of the car, which are closely correlated with each other. A linear regression model or a kNN based on the MSRP, mileage, age, make/model, and other smaller attributes can achieve an accurate prediction on the depreciation of a given vehicle (R^2 = ~0.86).

I think I am now permanently banned from TrueCar - I cannot access the website at all from my home IP. I will be doing future web scraping projects using a VPN or proxy from now on. I learned the importance of caching the HTML webpages, in case I wanted to make changes to my parser later. I also learned how to better spot duplicate listings, as most craigslist listings had no VINs or other IDs attached.

The limitations of the dataset and models are: the asking price isn't the sale price, 61% of the listings are from NJ or PA, only Craigslist has the “text flags,” and the data is from this week.

The hardest part for me was writing this report. I’ve always struggled in writing reports. The web scraping was also difficult, I had to change sites several times as some websites are not friendly for web scraping.

AI Usage Statement (4%)

- I used Claude only to write the code. To be clear, I give it instructions on how to do it, and review all the code it writes. I am not just having it generate the entire project.
- Claude also went through the data and came up with the set of rules(typos, placeholder prices, reposts, price being far from similar cars, etc) and did the attribute statistics table.
- Also learned the Pipeline object through Claude